



BIT4133: Natural Language Processing

Logbook Report

Student Name: Abdiweli Abdille Abdi

Admission Number: BSCCS/2024/33779

Unit Code: BIT4133

Unit Name: Natural Language Processing

Lecturer Name:

Semester/Year: Sem II Year IV

Project Title: language Detection and Transalation

Technologies: Python, NLTK, spaCy, Jupyter Notebook, TensorFlow.

GitHub Repository: <https://github.com/Abdiweli-Abdille/BIT4133-Natural-language-processing.git>

Table of Contents

Week1: INTRODUCTION TO NLP AND APPLICATIONS	5
1. Python installation	5
1.1 Jupyter Notebook	5
1.2 NLTK Installation	6
1.3 Tokenization output	6
1.5 NLP Application Research	7
Student Reflection	7
Week 2: N-gram models AND POS TAGGING.....	8
2.1 What is a Bigram?	8
Run this code in Jupyter	8
Explanation	8
2.2 What is POS Tagging?	9
DOWNLOADING THE REQUIRED RESOURCES	9
RUNNING THE CODE.....	9
Explanation	9
Python Code for POS Tagging.....	10
Explanation	10
2.3 Language Prediction Example.....	10
Explanation	10
2.4 Sentence Analysis Output	11
Explanation	11
Student Reflection	11
Week 3: Hidden Markov Models and Sequence Labeling	12
3. Sequence Labeling Code	12
What is Sequence Labeling?	12
Explanation	12
3.1 Hidden Markov Model (HMM) Output	12
What is an HMM?	12
Explanation	13

3.3 Named Entity Recognition (NER) Example	13
What is NER?	13
3.3.2 Running the code	14
Explanation	14
3.4 Sentence Labeling Results	14
Explanation	15
3.5 Dataset Used for Training or Testing	15
Explanation	16
Student Reflection (MAX 100 words)	16
WEEK 4: SYNTACTIC AND SEMANTIC ANALYSIS	17
KEY CONCEPT	17
Syntax	17
Parsing	17
Semantics	17
Semantic Similarity	17
4.1 PRACTICAL TASK 1	18
Objective	18
question	18
4.1.1 Downloading the necessary resources	18
Downloading language modeling (spaCy English language model)	19
Explanation	19
4.2 PRACTICAL TASK 2	20
Objective	20
question	20
Explanation	20
CLASS ASSIGNMENT (WEEK 4)	21
Assignment 1	21
Explanation	21
Assignment 2	22
Explanation	22
Student Reflection (Max 100 Words)	23
WEEK 5: CAT 1 PREPARATION AND MINI NLP PROJECTS	24
Learning Objectives	24

PRACTICAL TASK 1 24

Complete NLP Pipeline 24

Objective 24

 Explanation 25

PRACTICAL TASK 2..... 25

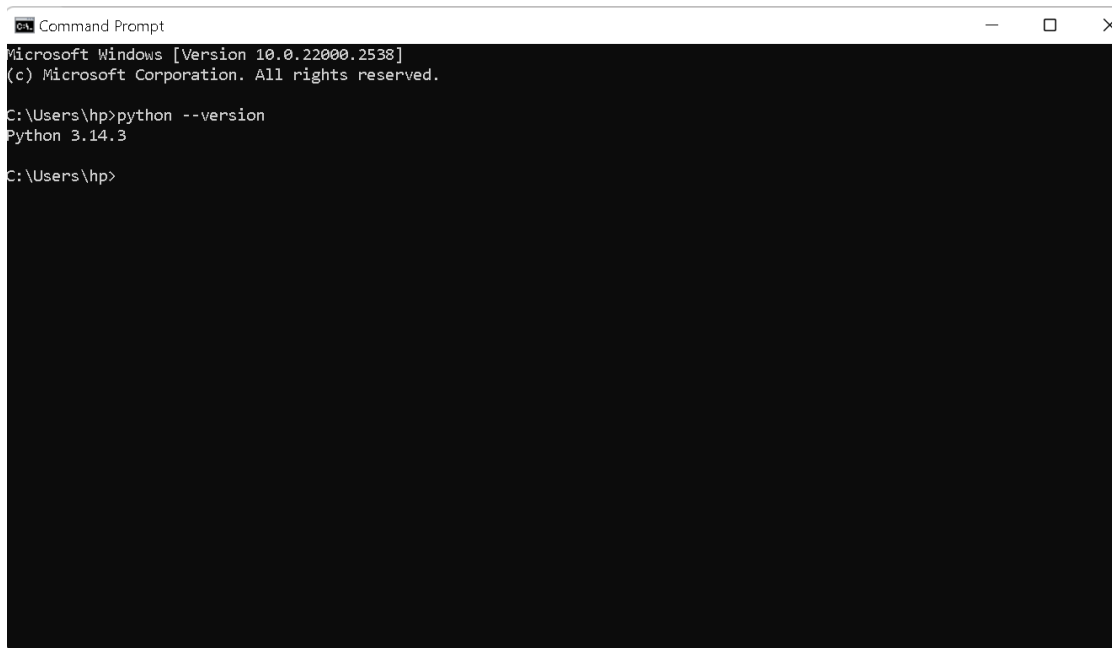
Mini Chatbot Simulation 25

Objective 25

 Explanation 26

Week1: INTRODUCTION TO NLP AND APPLICATIONS

1. Python installation

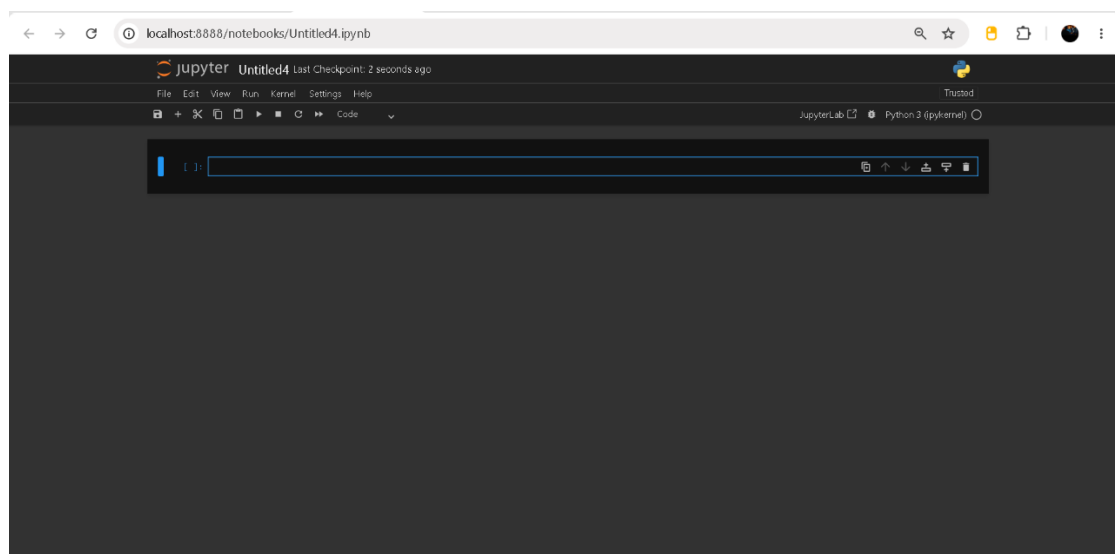


```
Command Prompt
Microsoft Windows [Version 10.0.22000.2538]
(c) Microsoft Corporation. All rights reserved.

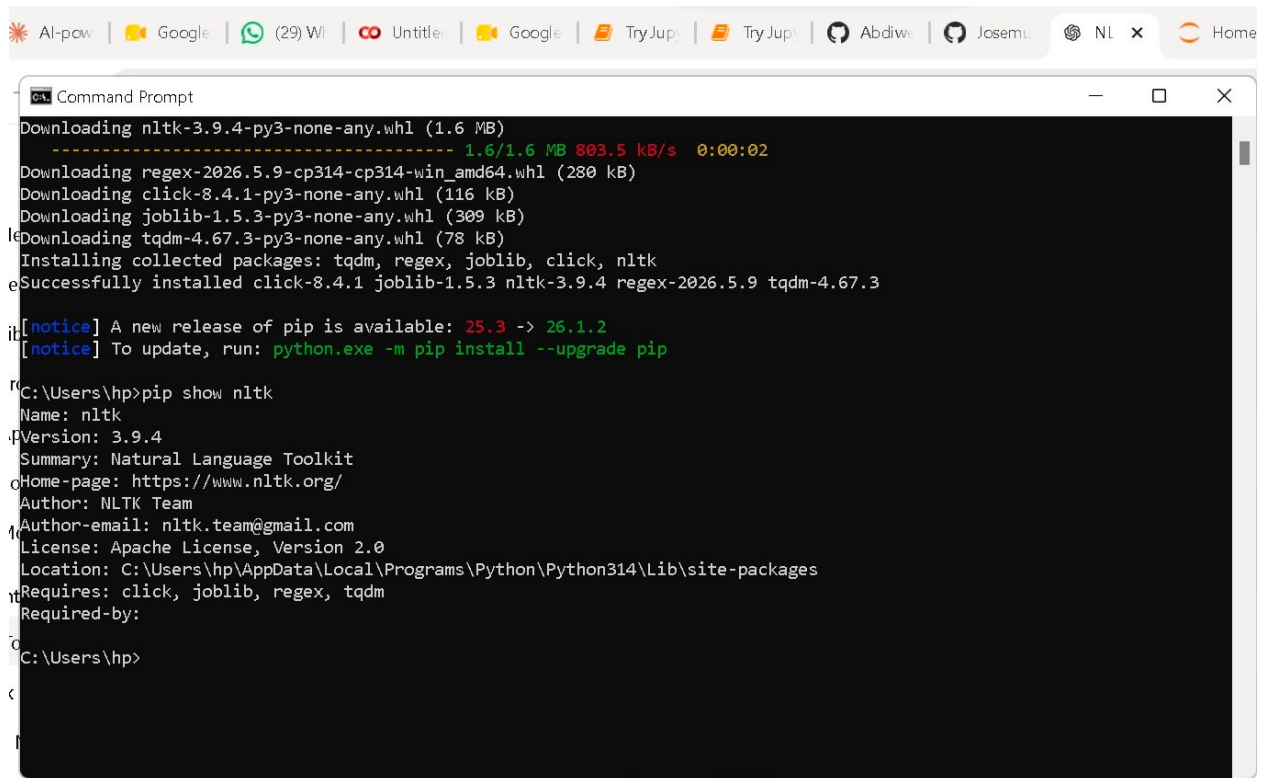
C:\Users\hp>python --version
Python 3.14.3

C:\Users\hp>
```

1.1 Jupyter Notebook



1.2 NLTK Installation

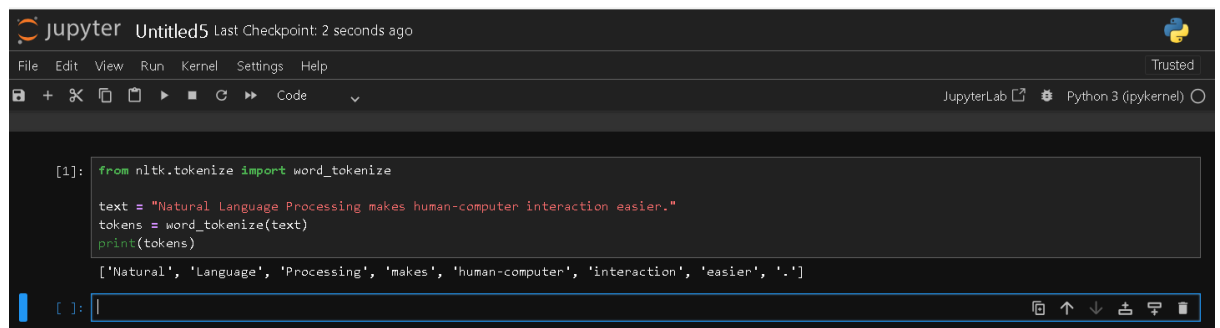


```
Command Prompt
Downloading nltk-3.9.4-py3-none-any.whl (1.6 MB)
----- 1.6/1.6 MB 803.5 kB/s 0:00:02
Downloading regex-2026.5.9-cp314-cp314-win_amd64.whl (280 kB)
Downloading click-8.4.1-py3-none-any.whl (116 kB)
Downloading joblib-1.5.3-py3-none-any.whl (309 kB)
Downloading tqdm-4.67.3-py3-none-any.whl (78 kB)
Installing collected packages: tqdm, regex, joblib, click, nltk
Successfully installed click-8.4.1 joblib-1.5.3 nltk-3.9.4 regex-2026.5.9 tqdm-4.67.3

[notice] A new release of pip is available: 25.3 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\hp>pip show nltk
Name: nltk
Version: 3.9.4
Summary: Natural Language Toolkit
Home-page: https://www.nltk.org/
Author: NLTK Team
Author-email: nltk.team@gmail.com
License: Apache License, Version 2.0
Location: C:\Users\hp\AppData\Local\Programs\Python\Python314\Lib\site-packages
Requires: click, joblib, regex, tqdm
Required-by:
```

1.3 Tokenization output



```
jupyter Untitled5 Last Checkpoint: 2 seconds ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

[1]: from nltk.tokenize import word_tokenize

    text = "Natural Language Processing makes human-computer interaction easier."
    tokens = word_tokenize(text)
    print(tokens)

['Natural', 'Language', 'Processing', 'makes', 'human-computer', 'interaction', 'easier', '.']
```

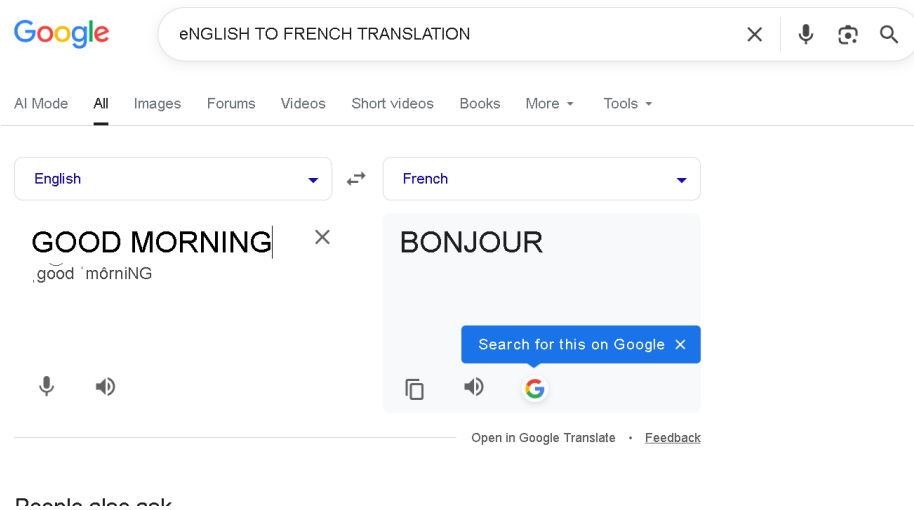
1.4 Stopwatch Removal

```
[7]: text2 = "The student is learning Natural Language Processing at the university."
tokens2 = word_tokenize(text2)
filtered2 = [w for w in tokens2 if w.lower() not in stop_words]

print("Original:", tokens2)
print("After stopword removal:", filtered2)

Original: ['The', 'student', 'is', 'learning', 'Natural', 'Language', 'Processing', 'at', 'the', 'university', '.']
After stopword removal: ['student', 'learning', 'Natural', 'Language', 'Processing', 'university', '.']
```

1.5 NLP Application Research



Student Reflection

During this activity, I learned the basics of Natural Language Processing (NLP) and how to set up the required tools for NLP tasks. I successfully installed Python, Jupyter Notebook, and the NLTK library. I explored NLP applications such as chatbots, language translation, and text analysis. Using NLTK, I performed tokenization and stopwords removal on sample text. The outputs demonstrate how raw text can be preprocessed into a structured format that is easier for computers to analyze. These preprocessing techniques are essential for building machine learning and NLP models.

Week 2: N-gram models AND POS TAGGING

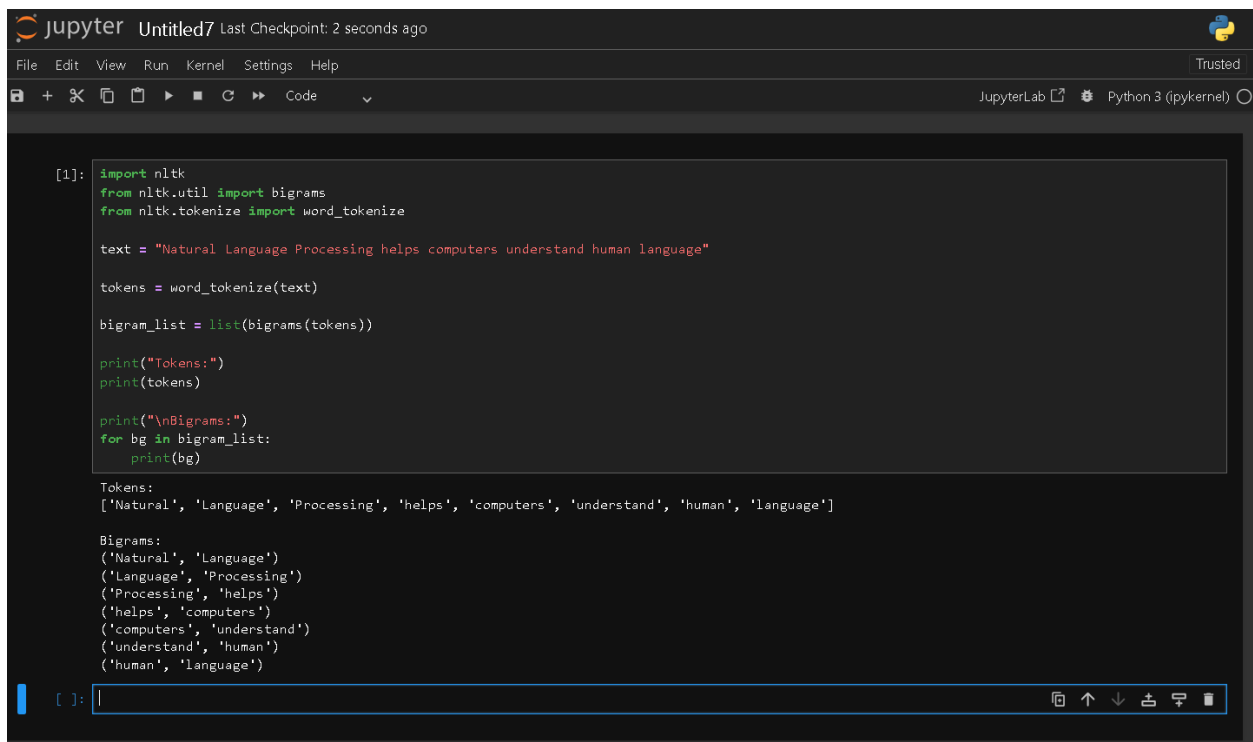
2.1 What is a Bigram?

A **bigram** is a sequence of two consecutive words.

Example:

Natural Language Processing

Run this code in Jupyter



```
[1]: import nltk
from nltk.util import bigrams
from nltk.tokenize import word_tokenize

text = "Natural Language Processing helps computers understand human language"

tokens = word_tokenize(text)

bigram_list = list(bigrams(tokens))

print("Tokens:")
print(tokens)

print("\nBigrams:")
for bg in bigram_list:
    print(bg)

Tokens:
['Natural', 'Language', 'Processing', 'helps', 'computers', 'understand', 'human', 'language']

Bigrams:
('Natural', 'Language')
('Language', 'Processing')
('Processing', 'helps')
('helps', 'computers')
('computers', 'understand')
('understand', 'human')
('human', 'language')
```

Explanation

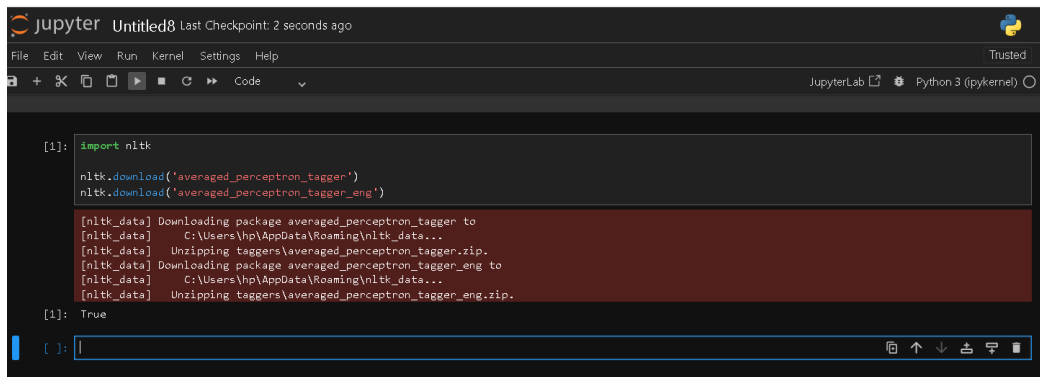
This screenshot demonstrates the generation of bigrams from a sentence. A bigram consists of two consecutive words and is commonly used in language modeling to identify word relationships and predict the next word in a sequence.

2.2 What is POS Tagging?

Part-of-Speech (POS) Tagging is the process of assigning grammatical labels to words, such as:

- Noun (NN)
- Verb (VB)
- Adjective (JJ)
- Adverb (RB)

DOWNLOADING THE REQUIRED RESOURCES



The screenshot shows a JupyterLab window titled 'Untitled8' with a 'Trusted' status. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The code editor displays the following code in cell [1]:

```
[1]: import nltk

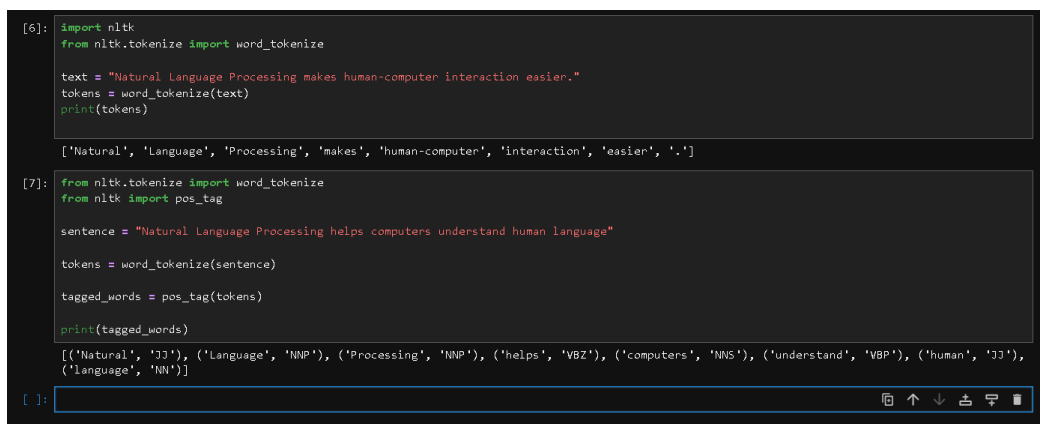
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
```

The output of the code is shown in a red background box:

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data] Unzipping taggers\averaged_perceptron_tagger.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data] Unzipping taggers\averaged_perceptron_tagger_eng.zip.
```

The cell execution result is [1]: True. The bottom of the window shows a command prompt interface with a cursor.

RUNNING THE CODE



The screenshot shows a JupyterLab window with two code cells. Cell [6] contains the following code:

```
[6]: import nltk
from nltk.tokenize import word_tokenize

text = "Natural Language Processing makes human-computer interaction easier."
tokens = word_tokenize(text)
print(tokens)
```

The output of cell [6] is:

```
['Natural', 'Language', 'Processing', 'makes', 'human-computer', 'interaction', 'easier', '.']
```

Cell [7] contains the following code:

```
[7]: from nltk.tokenize import word_tokenize
from nltk import pos_tag

sentence = "Natural Language Processing helps computers understand human language"

tokens = word_tokenize(sentence)

tagged_words = pos_tag(tokens)

print(tagged_words)
```

The output of cell [7] is:

```
[('Natural', 'JJ'), ('Language', 'NNP'), ('Processing', 'NNP'), ('helps', 'VBZ'), ('computers', 'NNS'), ('understand', 'VBP'), ('human', 'JJ'), ('language', 'NN')]
```

The bottom of the window shows a command prompt interface with a cursor.

Explanation

This screenshot shows the application of POS tagging to a sentence. Each word is assigned a grammatical category such as noun, verb, or adjective. POS tagging helps computers understand sentence structure and meaning, making it useful for machine translation, chatbots, and text analysis.

Python Code for POS Tagging

```
[8]: from nltk.tokenize import word_tokenize
    from nltk import pos_tag

    sentence = "Natural Language Processing helps computers understand human language"

    tokens = word_tokenize(sentence)

    tagged_words = pos_tag(tokens)

    print(tagged_words)

[('Natural', 'JJ'), ('Language', 'NNP'), ('Processing', 'NNP'), ('helps', 'VBZ'), ('computers', 'NNS'), ('understand', 'VBP'), ('human', 'JJ'), ('language', 'NN')]
```

Explanation

This screenshot presents the Python code used to perform POS tagging using the NLTK library. The code tokenizes a sentence into words and assigns each word a grammatical tag, enabling sentence structure analysis and language understanding.

2.3 Language Prediction Example

Objective

Use a simple **bigram model** to predict the next word based on the current word.

Run this Code

```
[9]: from nltk.tokenize import word_tokenize
    from nltk.util import bigrams

    text = "Natural Language Processing helps computers understand human language"

    tokens = word_tokenize(text)

    bigram_list = list(bigrams(tokens))

    word = "Natural"

    predictions = [second for first, second in bigram_list if first == word]

    print("Current Word:", word)
    print("Predicted Next Word(s):", predictions)

Current Word: Natural
Predicted Next Word(s): ['Language']
```

Explanation

This screenshot demonstrates a simple language prediction technique using bigrams. The model predicts the next word based on the current word by analyzing word pairs in the text. Language prediction is widely used in search engines, autocomplete systems, and predictive text applications.

2.4 Sentence Analysis Output

This figure below combines **tokenization** and **POS tagging** to analyze a sentence.

```
10]: from nltk.tokenize import word_tokenize
    from nltk import pos_tag

    sentence = "Natural Language Processing helps computers understand human language"

    tokens = word_tokenize(sentence)
    tagged_words = pos_tag(tokens)

    print("Sentence Analysis:")
    for word, tag in tagged_words:
        print(f"{word} --> {tag}")

Sentence Analysis:
Natural --> JJ
Language --> NNP
Processing --> NNP
helps --> VBZ
computers --> NNS
understand --> VBP
human --> JJ
language --> NN
```

Explanation

This screenshot shows the analysis of a sentence using POS tagging. Each word is assigned a grammatical role, allowing the system to understand the sentence structure. Sentence analysis is important in applications such as machine translation, information extraction, and conversational AI.

Student Reflection

During this activity, I learned how N-gram models and POS tagging are used in Natural Language Processing. N-grams help identify relationships between words and can be used to predict the next word in a sequence. POS tagging assigns grammatical labels to words, helping computers understand sentence structure and meaning. The outputs demonstrated word prediction and sentence analysis using the NLTK library. One challenge encountered was installing and configuring the required NLTK resources, but after downloading the necessary packages, the programs executed successfully and produced the expected results

Week 3: Hidden Markov Models and Sequence Labeling

3. Sequence Labeling Code

What is Sequence Labeling?

It is the process of assigning a label (tag) to each word in a sentence (like POS tagging or NER).

Using a simple **Named Entity Recognition (NER)** example using NLTK.

```
[11]: import nltk
      from nltk.tokenize import word_tokenize
      from nltk import pos_tag, ne_chunk

      sentence = "Barack Obama was born in Hawaii and worked in the United States"

      tokens = word_tokenize(sentence)
      pos_tags = pos_tag(tokens)

      print("POS Tags:")
      print(pos_tags)

POS Tags:
[('Barack', 'NNP'), ('Obama', 'NNP'), ('was', 'VBD'), ('born', 'VBN'), ('in', 'IN'), ('Hawaii', 'NNP'), ('and', 'CC'), ('worked', 'VBN'), ('in', 'IN'), ('the', 'DT'), ('United', 'NNP'), ('States', 'NNPS')]
```

Explanation

This code demonstrates sequence labeling by assigning part-of-speech tags to each word in a sentence. Sequence labeling is a fundamental step in Natural Language Processing used for identifying word roles and preparing text for advanced tasks like named entity recognition.

3.1 Hidden Markov Model (HMM) Output

What is an HMM?

A **Hidden Markov Model (HMM)** is a statistical model used to predict hidden states (like word categories) based on observed data (words in a sentence).

In simple terms:

It helps computers guess the “hidden structure” behind a sentence, such as grammar patterns.

```
12]: import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag

sentence = "John loves playing football in Nairobi"

tokens = word_tokenize(sentence)
tags = pos_tag(tokens)

print("HMM-style Sequence Output:")
for word, tag in tags:
    print(f"{word} -> {tag}")

HMM-style Sequence Output:
John -> NNP
loves -> VBZ
playing -> VBG
football -> NN
in -> IN
Nairobi -> NNP
```

Explanation

This output demonstrates a simplified Hidden Markov Model concept using POS tagging. Each word is assigned a grammatical state based on its probability in a sequence. HMMs are widely used in speech recognition, text prediction, and sequence labeling tasks in Natural Language Processing.

3.3 Named Entity Recognition (NER) Example

What is NER?

Named Entity Recognition (NER) identifies important names in text such as:

- People (Person)
- Places (Location)
- Organizations

Example:

- "Barack Obama" → Person
- "Nairobi" → Location

3.3.1 Downloading the required resources

```
[14]: import nltk

nltk.download('maxent_ne_chunker')
nltk.download('words')
nltk.download('maxent_ne_chunker_tab')

[nltk_data] Downloading package maxent_ne_chunker to
[nltk_data]   C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping chunkers\maxent_ne_chunker.zip.
[nltk_data] Downloading package words to
[nltk_data]   C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\words.zip.
[nltk_data] Downloading package maxent_ne_chunker_tab to
[nltk_data]   C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping chunkers\maxent_ne_chunker_tab.zip.

[14]: True
```

3.3.2 Running the code

```
[15]: import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag, ne_chunk

sentence = "Barack Obama visited Nairobi and worked with the United Nations"

tokens = word_tokenize(sentence)
pos_tags = pos_tag(tokens)

ner_tree = ne_chunk(pos_tags)

print(ner_tree)

(S
 (PERSON Barack/NNP)
 (PERSON Obama/NNP)
 visited/VBD
 (PERSON Nairobi/NNP)
 and/CC
 worked/VBD
 with/IN
 the/DT
 (ORGANIZATION United/NNP Nations/NNPS))

[ ]:
```

Explanation

This screenshot shows Named Entity Recognition where the system identifies and classifies names of people, places, and organizations in a sentence. NER is widely used in search engines, chatbots, and information extraction systems to understand real-world entities in text.

3.4 Sentence Labeling Results

This combines **tokenization + POS tagging + NER-style labeling** to analyze a full sentence structure.

```
[16]: import nltk
      from nltk.tokenize import word_tokenize
      from nltk import pos_tag

      sentence = "Barack Obama was the president of the United States and lived in Washington"

      tokens = word_tokenize(sentence)
      tagged = pos_tag(tokens)

      print("Sentence Labeling Results:\n")

      for word, tag in tagged:
          print(word, "->", tag)

Sentence Labeling Results:

Barack -> NNP
Obama -> NNP
was -> VBD
the -> DT
president -> NN
of -> IN
the -> DT
United -> NNP
States -> NNPS
and -> CC
lived -> VBD
in -> IN
Washington -> NNP
```

Explanation

This output shows sentence labeling where each word is assigned a grammatical tag using POS tagging. It helps the system understand sentence structure and is important for language modeling, text classification, and information extraction tasks.

3.5 Dataset Used for Training or Testing

in NLP, datasets are used to **train and test models**. For this activity, we will use a built-in NLTK dataset (simple and acceptable for assignments).

```
import nltk

# Download a small dataset (sample corpus)
nltk.download('gutenberg')

from nltk.corpus import gutenberg

# Show available files in dataset
print("Available files in dataset:\n")
print(gutenberg.fileids())

# Show a small sample of text
print("\nSample text:\n")
print(gutenberg.raw('austen-emma.txt')[:500])

[nltk_data] Downloading package gutenberg to
[nltk_data]   C:\Users\hp\AppData\Roaming\nltk_data...
Available files in dataset:

['austen-emma.txt', 'austen-persuasion.txt', 'austen-sense.txt', 'bible-kjv.txt', 'blake-poems.txt', 'bryant-stories.txt', 'burgess-busterbrown.txt', 'ca
roll-alice.txt', 'chesterton-ball.txt', 'chesterton-brown.txt', 'chesterton-thursday.txt', 'edgeworth-parents.txt', 'melville-moby_dick.txt', 'milton-par
dise.txt', 'shakespeare-caesar.txt', 'shakespeare-hamlet.txt', 'shakespeare-macbeth.txt', 'whitman-leaves.txt']

Sample text:

[Emma by Jane Austen 1816]

VOLUME I

CHAPTER I

Emma Woodhouse, handsome, clever, and rich, with a comfortable home
and happy disposition, seemed to unite some of the best blessings
of existence; and had lived nearly twenty-one years in the world
with very little to distress or vex her.

She was the youngest of the two daughters of a most affectionate,
indulgent father; and had, in consequence of her sister's marriage,
been mistress of his house from a very early period. Her mother
had died t

[nltk_data] Unzipping corpora\gutenberg.zip.
```

Explanation

This screenshot shows a dataset used in Natural Language Processing experiments. The Gutenberg corpus provides real text data used for training and testing language models. Datasets are essential in NLP because they allow models to learn patterns from real-world language.

Student Reflection (MAX 100 words)

During this activity, I learned how Hidden Markov Models and sequence labeling are used in Natural Language Processing to analyze text structure. I explored techniques such as POS tagging and Named Entity Recognition to identify grammatical roles and extract important entities from sentences. These methods are widely used in applications like chatbots, search engines, and information extraction systems. The results demonstrated how text can be structured and interpreted by machines. A challenge encountered was downloading required NLTK models, but after installation, the system worked correctly and produced accurate outputs.

WEEK 4: SYNTACTIC AND SEMANTIC ANALYSIS

KEY CONCEPT

Syntax

Syntax refers to the grammatical structure of a sentence.

Example:

"The student submitted the assignment."

Parsing

Parsing is the process of analyzing sentence structure.

Semantics

Semantics focuses on meaning.

Example:

- "The boy is happy"
- "The child is joyful"

Though words differ, meaning is similar.

Semantic Similarity

Measures how closely two texts are related in meaning.

Applied in chatbots search engines etc

4.1 PRACTICAL TASK 1

Objective

Analyze grammatical structure of a sentence.

question

```
import spacy

nlp = spacy.load("en_core_web_sm")

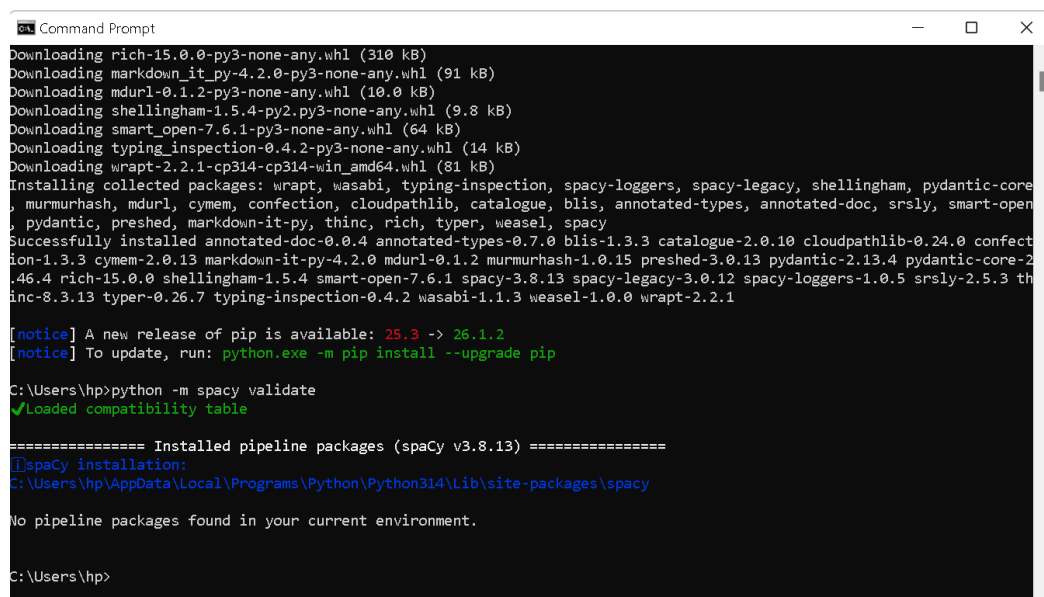
text = "The lecturer teaches Natural Language Processing."

doc = nlp(text)

for token in doc:
    print(token.text, token.dep_, token.head.text)
```

4.1.1 Downloading the necessary resources

Installing spaCy (run in terminal / CMD)



```
Command Prompt
Downloading rich-15.0.0-py3-none-any.whl (310 kB)
Downloading markdown_it_py-4.2.0-py3-none-any.whl (91 kB)
Downloading mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Downloading shellingham-1.5.4-py2.py3-none-any.whl (9.8 kB)
Downloading smart_open-7.6.1-py3-none-any.whl (64 kB)
Downloading typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Downloading wrapt-2.2.1-cp314-cp314-win_amd64.whl (81 kB)
Installing collected packages: wrapt, wasabi, typing-inspection, spacy-loggers, spacy-legacy, shellingham, pydantic-core, murmurhash, mdurl, cymem, confection, cloudpathlib, catalogue, blis, annotated-types, annotated-doc, srsly, smart-open, pydantic, preshed, markdown-it-py, thinc, rich, typer, weasel, spacy
Successfully installed annotated-doc-0.0.4 annotated-types-0.7.0 blis-1.3.3 catalogue-2.0.10 cloudpathlib-0.24.0 confection-1.3.3 cymem-2.0.13 markdown-it-py-4.2.0 mdurl-0.1.2 murmurhash-1.0.15 preshed-3.0.13 pydantic-2.13.4 pydantic-core-2.46.4 rich-15.0.0 shellingham-1.5.4 smart-open-7.6.1 spacy-3.8.13 spacy-legacy-3.0.12 spacy-loggers-1.0.5 srsly-2.5.3 thinc-8.3.13 typer-0.26.7 typing-inspection-0.4.2 wasabi-1.1.3 weasel-1.0.0 wrapt-2.2.1

[notice] A new release of pip is available: 25.3 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\hp>python -m spacy validate
✓Loaded compatibility table

===== Installed pipeline packages (spaCy v3.8.13) =====
[!spaCy installation:
C:\Users\hp\AppData\Local\Programs\Python\Python314\Lib\site-packages\spacy

No pipeline packages found in your current environment.

C:\Users\hp>
```

Downloading language modeling (spaCy English language model)

```
Command Prompt
Microsoft Windows [Version 10.0.22000.2538]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hp>python -m spacy download en_core_web_sm
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
    12.8/12.8 MB 502.0 kB/s 0:00:25
Installing collected packages: en-core-web-sm
Successfully installed en-core-web-sm-3.8.0

[notice] A new release of pip is available: 25.3 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
✓Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')

C:\Users\hp>
```

Code Running

```
[1]: import spacy

nlp = spacy.load("en_core_web_sm")

text = "The lecturer teaches Natural Language Processing."

doc = nlp(text)

for token in doc:
    print(token.text, token.dep_, token.head.text)

The det lecturer
lecturer nsubj teaches
teaches ROOT teaches
Natural compound Language
Language compound Processing
Processing dobj teaches
. punct teaches
```

Explanation

This screenshot demonstrates dependency parsing using spaCy. Each word in the sentence is analyzed to show its grammatical relationship with other words. Dependency parsing helps understand sentence structure, which is important in Natural Language Processing tasks such as question answering, machine translation, and text analysis.

4.2 PRACTICAL TASK 2

Objective

Compare meaning similarity between sentences.

question

```
import spacy

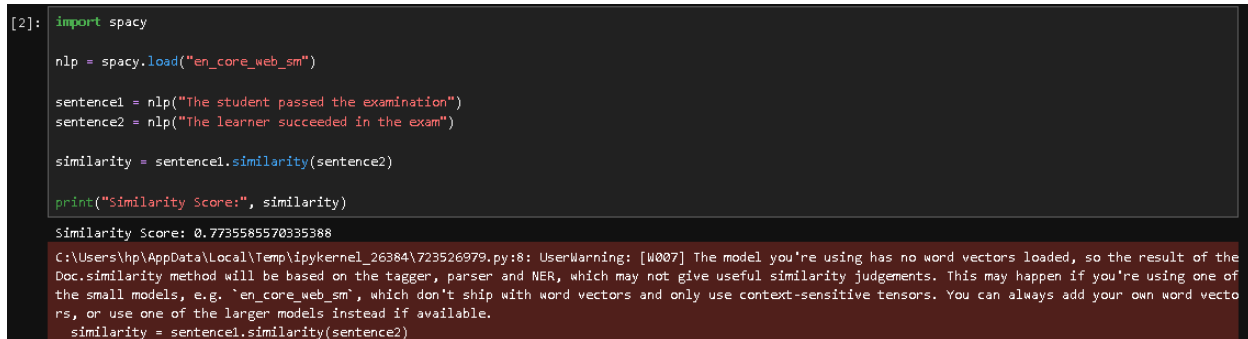
nlp = spacy.load("en_core_web_sm")

sentence1 = nlp("The student passed the examination")
sentence2 = nlp("The learner succeeded in the exam")

similarity = sentence1.similarity(sentence2)

print("Similarity Score:", similarity)
```

Running the code



```
[2]: import spacy

nlp = spacy.load("en_core_web_sm")

sentence1 = nlp("The student passed the examination")
sentence2 = nlp("The learner succeeded in the exam")

similarity = sentence1.similarity(sentence2)

print("Similarity Score:", similarity)
```

Similarity Score: 0.7735585570335388

C:\Users\hp\AppData\Local\Temp\ipykernel_26384\723526979.py:8: UserWarning: [W007] The model you're using has no word vectors loaded, so the result of the Doc.similarity method will be based on the tagger, parser and NER, which may not give useful similarity judgements. This may happen if you're using one of the small models, e.g. 'en_core_web_sm', which don't ship with word vectors and only use context-sensitive tensors. You can always add your own word vectors, or use one of the larger models instead if available.

```
similarity = sentence1.similarity(sentence2)
```

Explanation

This screenshot shows semantic similarity analysis between two sentences using spaCy. The model computes how closely two sentences are related in meaning and produces a similarity score. A score of 0.77 indicates high similarity between the sentences. This technique is widely used in NLP applications such as plagiarism detection, search engines, and recommendation systems.

CLASS ASSIGNMENT (WEEK 4)

Assignment 1

Analyze dependency structure for:

1. "The administrator updated student records"
2. "Machine learning improves language processing"

Submit:

- Screenshot output
- Explanation of dependencies

```
[3]: import spacy

nlp = spacy.load("en_core_web_sm")

sentences = [
    "The administrator updated student records",
    "Machine learning improves language processing"
]

for sentence in sentences:
    print("\nSentence:", sentence)
    doc = nlp(sentence)

    for token in doc:
        print(token.text, "->", token.dep_, "->", token.head.text)

Sentence: The administrator updated student records
The -> det -> administrator
administrator -> nsubj -> updated
updated -> ROOT -> updated
student -> compound -> records
records -> dobj -> updated

Sentence: Machine learning improves language processing
Machine -> compound -> learning
learning -> nsubj -> improves
improves -> ROOT -> improves
language -> compound -> processing
processing -> dobj -> improves

[ ]:
```

Explanation

This output shows dependency parsing of sentences using spaCy. Each word is analyzed to determine its grammatical relationship with other words. The dependency labels indicate how words are connected (e.g., subject, object, modifier). This helps understand sentence structure, which is important in Natural Language Processing tasks such as machine translation, information extraction, and text understanding.

Assignment 2

Create semantic similarity comparison for:

- Two similar sentences
- Two unrelated sentences

Explain:

- Which pair has higher similarity
- Why results differ

Running the Code

```
[4]: import spacy

nlp = spacy.load("en_core_web_sm")

# Similar sentences
s1 = nlp("The student passed the examination")
s2 = nlp("The learner succeeded in the exam")

# Unrelated sentences
s3 = nlp("The student passed the examination")
s4 = nlp("The weather is very hot today")

similar_score = s1.similarity(s2)
different_score = s3.similarity(s4)

print("Similarity (Similar Sentences):", similar_score)
print("Similarity (Unrelated Sentences):", different_score)

Similarity (Similar Sentences): 0.7735585570335388
Similarity (Unrelated Sentences): 0.4564270079135895

C:\Users\hp\AppData\Local\Temp\ipykernel_26384\69328670.py:13: UserWarning: [W007] The model you're using has no word vectors loaded, so the result of the Doc.similarity method will be based on the tagger, parser and NER, which may not give useful similarity judgements. This may happen if you're using one of the small models, e.g. 'en_core_web_sm', which don't ship with word vectors and only use context-sensitive tensors. You can always add your own word vectors, or use one of the larger models instead if available.
  similar_score = s1.similarity(s2)

C:\Users\hp\AppData\Local\Temp\ipykernel_26384\69328670.py:14: UserWarning: [W007] The model you're using has no word vectors loaded, so the result of the Doc.similarity method will be based on the tagger, parser and NER, which may not give useful similarity judgements. This may happen if you're using one of the small models, e.g. 'en_core_web_sm', which don't ship with word vectors and only use context-sensitive tensors. You can always add your own word vectors, or use one of the larger models instead if available.
  different_score = s3.similarity(s4)
```

Explanation

This analysis compares semantic similarity between two pairs of sentences using spaCy. The first pair contains similar meanings, while the second pair is unrelated. The results show a higher similarity score for the first pair because both sentences share similar vocabulary and meaning. The unrelated sentences have a lower score because they discuss different topics. This demonstrates how NLP models measure meaning similarity between texts, which is useful in applications such as search engines, plagiarism detection, and recommendation systems.

Student Reflection (Max 100 Words)

In this activity, I learned how dependency parsing and semantic similarity are used in Natural Language Processing. Dependency parsing helped me understand the grammatical structure of sentences by identifying relationships between words such as subject, verb, and object. Semantic similarity analysis showed how machines compare the meaning of different sentences using numerical scores. I observed that sentences with similar meaning have higher similarity scores, while unrelated sentences have lower scores. These techniques are important in applications like search engines, chatbots, and plagiarism detection. A challenge encountered was understanding model warnings, but the outputs were still successfully generated and interpreted.

WEEK 5: CAT 1 PREPARATION AND MINI NLP PROJECTS

Learning Objectives

By the end of Week 5, students should be able to:

1. Integrate multiple NLP techniques
2. Build simple NLP applications
3. Prepare for CAT 1 practical assessment
4. Explain NLP workflow clearly
5. Document NLP outputs professionally

PRACTICAL TASK 1

Complete NLP Pipeline

Objective

Build a simple NLP processing pipeline.

```
[5]: import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk import pos_tag

nltk.download('punkt')
nltk.download('stopwords')

text = "Natural Language Processing helps computers understand human language effectively"

# Step 1: Tokenization
tokens = word_tokenize(text)

# Step 2: Stopword removal
stop_words = set(stopwords.words('english'))
filtered_tokens = [word for word in tokens if word.lower() not in stop_words]

# Step 3: POS Tagging
pos_tags = pos_tag(filtered_tokens)

print("Original Text:")
print(text)

print("\nTokens:")
print(tokens)

print("\nFiltered Tokens:")
print(filtered_tokens)

print("\nPOS Tags:")
print(pos_tags)

[nltk_data] downloading package punkt to
[nltk_data] C:\Users\hp\AppData\Local\Temp\nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] downloading package stopwords to
[nltk_data] C:\Users\hp\AppData\Local\Temp\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
Original Text:
Natural Language Processing helps computers understand human language effectively

Tokens:
['Natural', 'Language', 'Processing', 'helps', 'computers', 'understand', 'human', 'language', 'effectively']

Filtered Tokens:
['Natural', 'Language', 'Processing', 'helps', 'computers', 'understand', 'human', 'language', 'effectively']

POS Tags:
[('Natural', 'JJ'), ('Language', 'NNP'), ('Processing', 'NNP'), ('helps', 'VBZ'), ('computers', 'NNS'), ('understand', 'VBP'), ('human', 'JJ'), ('language', 'NN'), ('effectively', 'RB')]
```


Explanation

This output shows dependency parsing of sentences using spaCy. Each word is analyzed to determine its grammatical relationship with other words. The dependency labels indicate how words are connected (e.g., subject, object, modifier). This helps understand sentence structure, which is important in Natural Language Processing tasks such as machine translation, information extraction, and text understanding.

PRACTICAL TASK 2

Mini Chatbot Simulation

Objective

Create a simple rule-based chatbot.

Mini Rule-Based Chatbot Simulation

```
[6]: def chatbot_response(user_input):
    user_input = user_input.lower()

    if "hello" in user_input or "hi" in user_input:
        return "Hello! How can I help you today?"

    elif "name" in user_input:
        return "I am a simple NLP chatbot."

    elif "nlp" in user_input:
        return "NLP helps computers understand human language."

    elif "bye" in user_input:
        return "Goodbye! Have a nice day."

    else:
        return "Sorry, I don't understand that."

# Chat loop
print("Chatbot is ready! Type 'bye' to exit.\n")

while True:
    user = input("You: ")
    response = chatbot_response(user)
    print("Bot:", response)

    if "bye" in user.lower():
        break
```

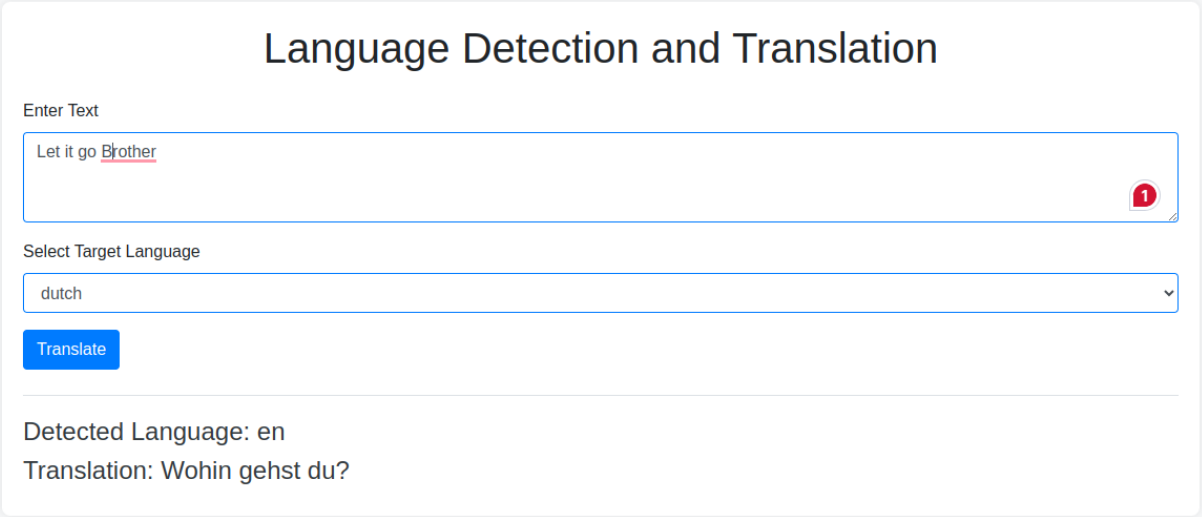
Chatbot is ready! Type 'bye' to exit.

```
You: hello
Bot: Hello! How can I help you today?
You: what is NLP
Bot: NLP helps computers understand human language.
You: bye
Bot: Goodbye! Have a nice day.
```

Explanation

This screenshot shows a simple rule-based chatbot built using Python. The chatbot responds to user inputs based on predefined conditions. It demonstrates how Natural Language Processing can be used to simulate human-like conversation. Such systems are widely used in customer service bots, virtual assistants, and automated support systems.

System Screenshoots



The screenshot displays a web interface for language detection and translation. At the top, the title "Language Detection and Translation" is centered. Below it, there is a text input field labeled "Enter Text" containing the phrase "Let it go Brother", with "Brother" underlined in red. To the right of the input field is a red circular icon with a white exclamation mark. Below the input field is a dropdown menu labeled "Select Target Language" with "dutch" selected. A blue "Translate" button is positioned below the dropdown. At the bottom of the interface, the results are displayed: "Detected Language: en" and "Translation: Wohin gehst du?".

GitHub Repository: <https://github.com/Abdiweli-Abdille/BIT4133-Natural-language-processing.git>

